

Project Report

Aerial Target Classification using a Base CNN and Transfer Learning

1. Introduction and Problem Reframing

The assigned problem calls for detection and classification of aerial targets: fighter jets, helicopters, drones, and rockets, using a base CNN together with at least three transfer learning models, evaluated with standard classification metrics: accuracy, precision, recall, sensitivity, specificity, F1 score, and ROC-AUC. The dataset referenced in the project brief, however, is the “Air Defense Object Detection Dataset (YOLOv8)”, which is annotated for object detection: every image is accompanied by YOLO-format bounding boxes and a class ID for each target instance (four classes: f16, helicopter, quadcopter, rocket), rather than a single image-level class label.

This creates a mismatch between the assignment's evaluation requirements and the dataset's native format. Accuracy, precision, recall, sensitivity, specificity, F1, and ROC-AUC are all classification metrics defined over a single predicted label per sample; they are not the metrics used to score an object detector, which is instead evaluated with localization-aware measures such as mean Average Precision (mAP) and IoU-based matching. Rather than force detection-style outputs into classification metrics they were not designed for, we converted the problem at the data level: every labelled bounding box was cropped out of its parent image using the provided YOLO annotations, resized to 224×224, and saved into per-class folders. This turns the object-detection dataset into a clean, single-label, 4-class image classification dataset, which is exactly the problem the requested metrics are built to score, and lets the base CNN and transfer learning models be compared on equal, well-defined footing. Section 2.1 gives the full details of this conversion.

2. Methodology

2.1 Dataset and Detection-to-Classification Conversion

The source dataset was located automatically from its data.yaml file and contains four target classes: f16, helicopter, quadcopter, and rocket. Images were first split at the image level into train (70%), validation (15%), and test (15%) sets: 6,318 training, 1,354 validation, and 1,355 test images, so that crops from the same source photo never leak across splits. Each YOLO label file was then parsed, its normalized (x_center , y_center , width, height) box converted to absolute pixel coordinates, clipped to the image bounds, cropped out, and resized to $224 \times 224 \times 3$. Because a single image can contain more than one labelled target, the number of resulting crops is larger than the number of source images: 7,453 training crops, 1,603 validation crops, and 1,598 test crops, organised into class-named sub-folders suitable for Keras' `flow_from_directory`.

2.2 Data Loading and Augmentation

Four separate sets of Keras ImageDataGenerator pipelines were built from the same underlying crops, one for the base CNN and one for each transfer learning model, so that every model could receive its own correct preprocessing (Section 2.4) while sharing an identical augmentation policy for a fair comparison. The training generator applies rotation ($\pm 20^\circ$), horizontal flips, zoom ($\pm 20\%$), and brightness jitter ($0.8\text{--}1.2\times$); validation and test generators apply only the model-specific preprocessing, with no augmentation. Class weights were computed on the training split (`compute_class_weight('balanced', ...)`) and passed to every training run to offset any residual class imbalance left after cropping.

2.3 End-to-End Pipeline

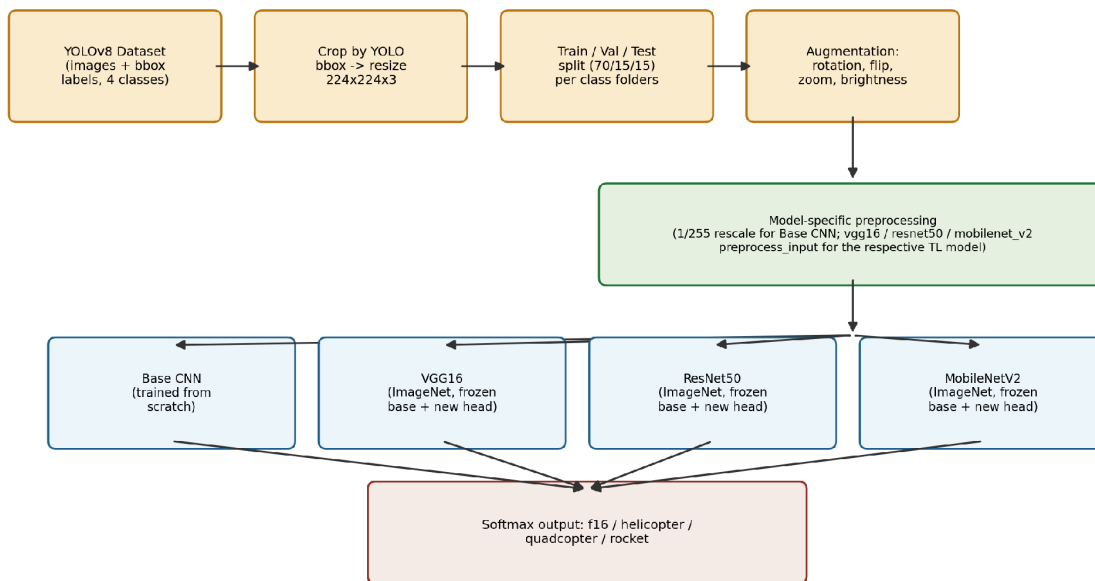


Figure 1. End-to-end pipeline: raw YOLOv8 annotations are cropped into a 4-class classification dataset, split, augmented, routed through per-model preprocessing, and fed into four independently trained classifiers.

2.4 Model Architectures

Four models were trained and evaluated on identical 224×224 crops with the same train/validation/test split:

- Base CNN: built from scratch (no pretrained weights): four convolutional blocks (32→256 filters, 3×3 kernels, ReLU, each followed by 2×2 max-pooling), then global average pooling, a 128-unit dense layer, dropout, and a 4-way softmax head. Total trainable parameters: 421,828.
- VGG16: ImageNet-pretrained convolutional base, frozen, with a new head (global average pooling → Dense(128, ReLU) → Dropout → softmax) trained on top. Trainable parameters: 66,180.
- ResNet50: same frozen-base, new-head setup as VGG16. Trainable parameters: 262,788.
- MobileNetV2: same frozen-base, new-head setup. Trainable parameters: 164,484.

All three transfer learning models keep their pretrained backbone entirely frozen and train only the newly added classification head; none of them are fine-tuned. This is a deliberate choice rather than an oversight: our training set of roughly 7,500 crops is small relative to the millions of images the backbones were pretrained on, and unfreezing the backbone risks two related problems: the head-only gradients being too noisy to fine-tune millions of pretrained weights without overfitting, and updating the frozen BatchNorm layers' running statistics on a much smaller and differently-distributed batch of data, which can destabilise features that were otherwise well-calibrated for ImageNet-style inputs. Frozen-backbone transfer learning with a lightweight trainable head is a standard and defensible baseline for exactly this small-data regime, and it is the configuration used for all three transfer learning models here.

2.5 Per-Model Preprocessing

Each model is paired with its own correct Keras preprocessing function: `vgg16.preprocess_input`, `resnet50.preprocess_input`, and `mobilenet_v2.preprocess_input` for the three transfer learning models, and a simple 1/255 rescale for the base CNN, rather than a single shared rescale applied uniformly to all four models. This is not a cosmetic detail. Each ImageNet-pretrained backbone was trained with a specific input normalisation (VGG16 and ResNet50 use channel-wise mean subtraction in BGR order with no [0,1] rescaling; MobileNetV2 scales inputs to [-1, 1]), and their frozen BatchNorm layers store running mean/variance statistics that are calibrated to that exact normalisation. In an earlier version of this pipeline we applied a shared 1/255 rescale to every model, including the transfer learning ones. VGG16 and MobileNetV2 tolerated the mismatch reasonably well, but ResNet50's accuracy collapsed to 0.34, close to random guessing on a 4-class problem, because its frozen BatchNorm statistics were calibrated on ImageNet's specific normalisation and broke down under the mismatched [0,1] input distribution. Switching ResNet50 (and every other transfer learning model) to its own dedicated `preprocess_input` function fixed this immediately and brought ResNet50 up to 0.84 accuracy, making it the best-performing model overall (Section 3). This is a genuine methodological finding worth stating explicitly: when a pretrained backbone is frozen, its preprocessing is part of the model, not an interchangeable data-loading detail.

2.6 Base CNN Hyperparameter Tuning

Because the transfer learning models' architecture is fixed by their pretrained design, tuning does not apply to them in the same way. The base CNN, in contrast, is trained entirely from scratch, so before the final run we ran a small grid search over learning rate (`{1e-3, 1e-4}`) and dropout rate (`{0.3, 0.5}`), training each of the four combinations for 5 epochs and selecting the configuration with the best validation accuracy. The results are summarised in Table 1.

Learning Rate	Dropout	Validation Accuracy (5 epochs)
1e-3	0.3	0.5508
1e-3	0.5	0.5434
1e-4	0.3	0.4978
1e-4	0.5	0.4953

Table 1. Base CNN grid search over learning rate and dropout. LR = 1e-3, dropout = 0.3 gave the best 5-epoch validation accuracy and was used for the final, full-length training run.

A learning rate of 1e-3 clearly outperformed 1e-4 within the short 5-epoch search budget, and the lower dropout rate of 0.3 edged out 0.5 at that learning rate; LR = 1e-3 with dropout = 0.3 was therefore selected for the final base CNN training run reported in Section 3.

2.7 Training Configuration

All four models were trained with the Adam optimizer, categorical cross-entropy loss, a batch size of 32, and early stopping on validation loss (patience = 5 epochs, best weights restored). Table 2 summarises the final configuration and the number of epochs each model actually trained for before early stopping triggered.

Model	Input Size	Batch Size	Optimizer	Learning Rate	Dropout	Epochs Trained	Trainable Params
Base CNN	224x224x3	32	Adam	0.001	0.3	30	421,828
VGG16	224x224x3	32	Adam	0.001	0.5	20	66,180
ResNet50	224x224x3	32	Adam	0.001	0.5	21	262,788
MobileNetV2	224x224x3	32	Adam	0.001	0.5	5	164,484

Table 2. Final training configuration and trainable-parameter count for all four models (loss = categorical cross-entropy for all models).

3. Results

3.1 Training and Validation Curves

Figure 2 shows training and validation accuracy/loss curves for all four models. The base CNN trains slowly and plateaus around 0.70–0.73 accuracy with a visible but modest train/validation gap, consistent with learning features from scratch on a comparatively small dataset. VGG16 and ResNet50 both converge quickly and track their validation curves closely, reflecting the stability of a frozen, well-calibrated pretrained backbone. MobileNetV2 rises fastest of all but trains for only 5 epochs: its validation loss/accuracy begin diverging from the training curves from around epoch 3 onward, and early stopping (patience = 5) correctly halted training at epoch 5 once it detected this. This is discussed further in Section 3.4.

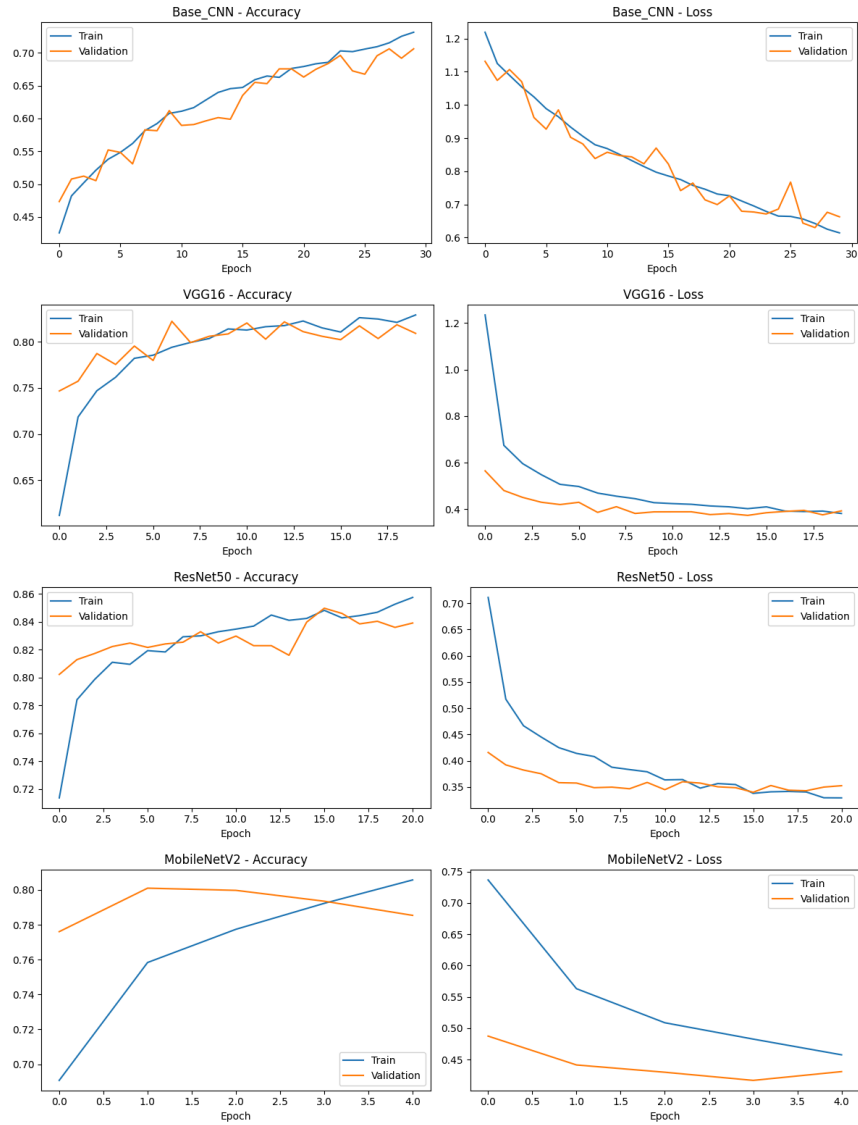


Figure 2. Training vs. validation accuracy and loss for Base CNN, VGG16, ResNet50, and MobileNetV2 (top to bottom).

3.2 Comparison of Evaluation Metrics

All four models were evaluated on the same held-out test set (1,598 crops). Table 3 reports accuracy, macro-averaged precision, recall (sensitivity), specificity, F1 score, and one-vs-rest ROC-AUC for each model.

Model	Accuracy	Macro Precision	Macro Recall (Sensitivity)	Macro Specificity	Macro F1	ROC-AUC (OVR)
Base CNN	0.7378	0.7202	0.7278	0.9149	0.7191	0.9351
VGG16	0.8179	0.7952	0.8029	0.9411	0.7949	0.9646
ResNet50	0.8423	0.8091	0.7961	0.9481	0.7992	0.9696
MobileNetV2	0.7916	0.7718	0.7790	0.9339	0.7652	0.9525

Table 3. Final test-set evaluation metrics for all four models (macro-averaged across the four classes).

3.3 ROC Curves

Figure 3 shows the per-class ROC curves for each model. Rocket and f16 are consistently the easiest classes to separate ($AUC \geq 0.96$ for every model), while helicopter and quadcopter are harder and drive most of the gap between models, their curves sit visibly closer to the diagonal for the base CNN than for the pretrained backbones.

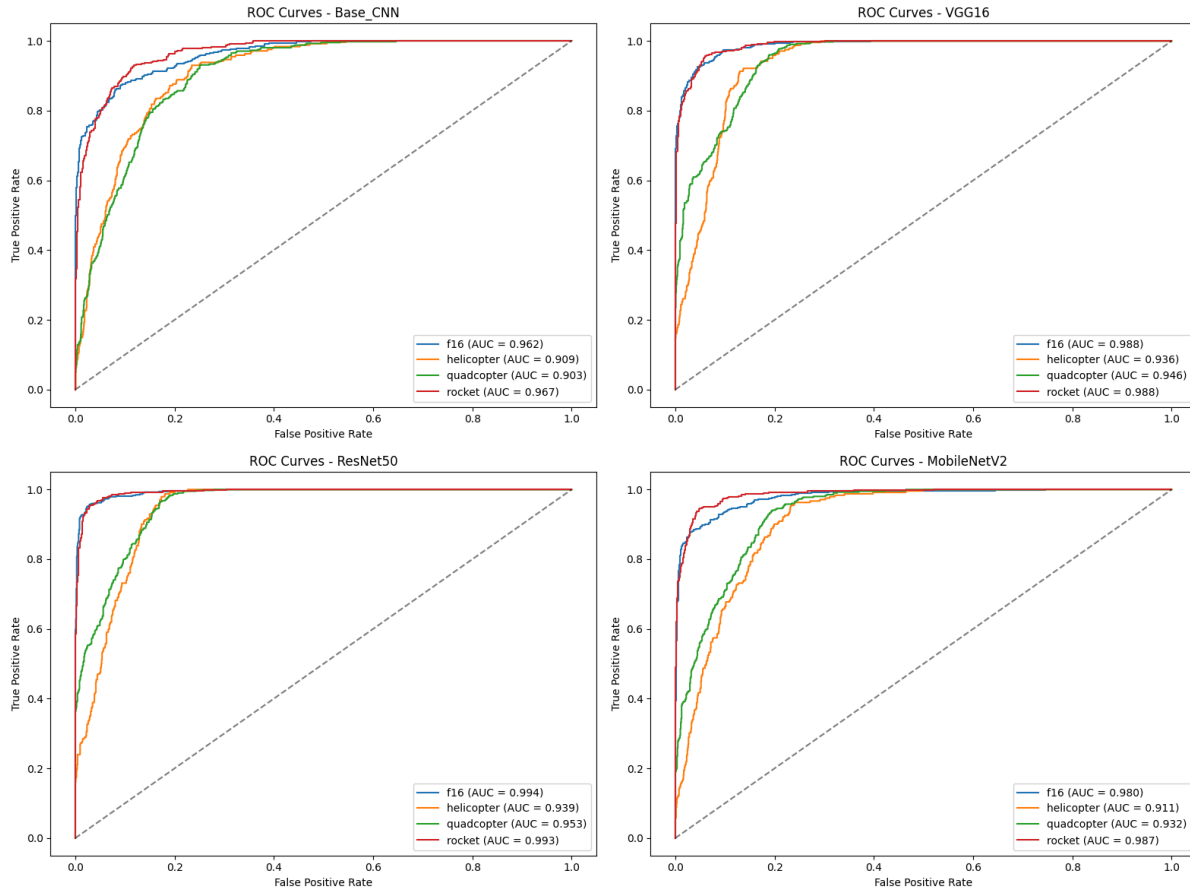


Figure 3. Per-class one-vs-rest ROC curves for Base CNN, VGG16, ResNet50, and MobileNetV2.

3.4 Discussion

Across every metric, the models rank ResNet50 > VGG16 > MobileNetV2 > Base CNN. This ordering is a useful sanity check rather than a coincidence: it matches what a stronger pretrained backbone would be expected to deliver over a lighter one, which in turn would be expected to beat a small convolutional network trained from scratch with no pretrained visual features to draw on. ResNet50's residual connections give it the richest feature representation of the three backbones, and once its preprocessing mismatch was corrected (Section 2.5) it edges out VGG16 on every metric except recall, where the two are close. MobileNetV2 sits between the transfer learning models and the base CNN, it benefits from ImageNet pretraining but has a much smaller classification head (164K trainable parameters, versus the larger backbone-adjacent heads of the other models) and simply has less trained capacity to work with.

MobileNetV2's early stopping after only 5 epochs (versus 20–30 for the other three models) is worth calling out explicitly because on its own it can look like a failure. The training curves in Figure 2 show this is not the case: MobileNetV2's smaller, lighter head converges much faster than the other models' heads, and its validation loss

and accuracy visibly start to diverge from the training curves from around epoch 3 onward, a classic overfitting signature. Early stopping (patience = 5, monitoring validation loss) is doing exactly what it is designed to do here: it caught the onset of overfitting and restored the best validation-loss weights rather than letting the model continue to memorise the training set. MobileNetV2's final test accuracy of 0.79 is entirely consistent with a model that was stopped at the right time, not one that broke.

5. Conclusion

Converting the YOLOv8 aerial-target detection dataset into a 4-class image classification dataset by cropping labelled bounding boxes let us evaluate a base CNN and three transfer learning models (VGG16, ResNet50, MobileNetV2) with the classification metrics the assignment requires. ResNet50 was the strongest model overall (0.842 accuracy, 0.970 ROC-AUC), followed by VGG16, MobileNetV2, and the from-scratch base CNN, an ordering consistent with the relative representational strength of each backbone. Two methodological details proved important to get right: pairing each pretrained backbone with its own preprocessing function (without which ResNet50's frozen BatchNorm statistics broke down entirely), and letting early stopping curtail MobileNetV2's training once its smaller head began to overfit rather than treating the resulting short training run as a failure.